

TCP Communication between ESP32 and Linux using Caesar Cipher

Carmen Nochipa Orozco Olivares

Professor: Emilio Tonix

Subject: Microprocessors

1. Build Instructions (Linux & ESP-IDF)

ESP-IDF Setup

- ESP-IDF version: 5.2
- Environment setup:

```
. $HOME/esp/esp-idf/export.sh
idf.py set-target esp32
idf.py build
idf.py -p /dev/ttyUSB0 flash
idf.py monitor
```

- Network configuration: updated SSID and password using:

```
idf.py menuconfig
# Example configuration:
# SSID: Nothing
# Password: *****
```

Linux Client Build

```
cd ~/Documents/Tarea6/caesar-tcp/linux
mkdir build && cd build
cmake ..
make
./tcp_client 10.70.31.76 3333 "Carmen_123"
```

Network Interface Check

```
ip a
# Output:
# wlo1: inet 10.70.31.118/24
```

ESP32 IP (server): **10.70.31.76**

Linux Client IP: **10.70.31.118**

2. Wireshark Screenshots

- Applied filter: `tcp.port == 3333`
- Observed TCP connection between 10.70.31.118 (client) and 10.70.31.76 (server).

- Payload shows Caesar-encrypted string.

No.	Time	Source	Destination	Protocol	Length	Info
133	31.589617966	10.70.31.118	10.70.31.76	TCP	74	35506 → 3333 [SYN] Seq=6
135	32.620606170	10.70.31.118	10.70.31.76	TCP	74	[TCP Retransmission] 355
139	33.644710712	10.70.31.118	10.70.31.76	TCP	74	[TCP Retransmission] 355
140	33.861625033	10.70.31.76	10.70.31.118	TCP	58	3333 → 35506 [SYN, ACK]
141	33.861699541	10.70.31.118	10.70.31.76	TCP	54	35506 → 3333 [ACK] Seq=1
142	33.861793872	10.70.31.118	10.70.31.76	TCP	65	35506 → 3333 [PSH, ACK]
143	34.324601338	10.70.31.118	10.70.31.76	TCP	65	[TCP Retransmission] 355
144	34.391175365	10.70.31.76	10.70.31.118	TCP	65	3333 → 35506 [PSH, ACK]
145	34.391202466	10.70.31.118	10.70.31.76	TCP	54	35506 → 3333 [ACK] Seq=1
146	34.391300724	10.70.31.118	10.70.31.76	TCP	54	35506 → 3333 [FIN, ACK]
147	34.393723912	10.70.31.76	10.70.31.118	TCP	54	3333 → 35506 [FIN, ACK]
148	34.393740936	10.70.31.118	10.70.31.76	TCP	54	35506 → 3333 [ACK] Seq=1
149	34.405032331	10.70.31.76	10.70.31.118	TCP	54	3333 → 35506 [ACK] Seq=1

Frame 133: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 0	0000	68 25 dd 30 22 90 5c 5f	67 63 9e 53 08 00 4
Ethernet II, Src: Intel_63:9e:53 (5c:5f:67:63:9e:53), Dst: 10.70.31.76 (08:00:00:08:00:08)	0010	00 3c 8b 31 40 00 40 06	5c 3d 0a 46 1f 76 0
Internet Protocol Version 4, Src: 10.70.31.118, Dst: 10.70.31.76	0020	1f 4c 8a b2 0d 05 cd 9a	7d 0c 00 00 00 00 a
Transmission Control Protocol, Src Port: 35506, Dst Port: 3333	0030	fa f0 53 7c 00 00 02 04	05 b4 04 02 08 0a e
	0040	2e 33 00 00 00 00 01 03	03 07

Figure 1: Wireshark capture filter showing TCP packets between ESP32 and Linux.

```
connect: No route to host
carmen@carmen-G3-3779:~/Documents/Tarea6/caesar-tcp/linux/build$ cd ~/Documents/
Tarea6/caesar-tcp/linux/build
./tcp_client 10.70.31.76 3333 "Carmen_123"
Respuesta 11 | Descifrado: Carmen_123
carmen@carmen-G3-3779:~/Documents/Tarea6/caesar-tcp/linux/build$
```

Figure 3: Terminal

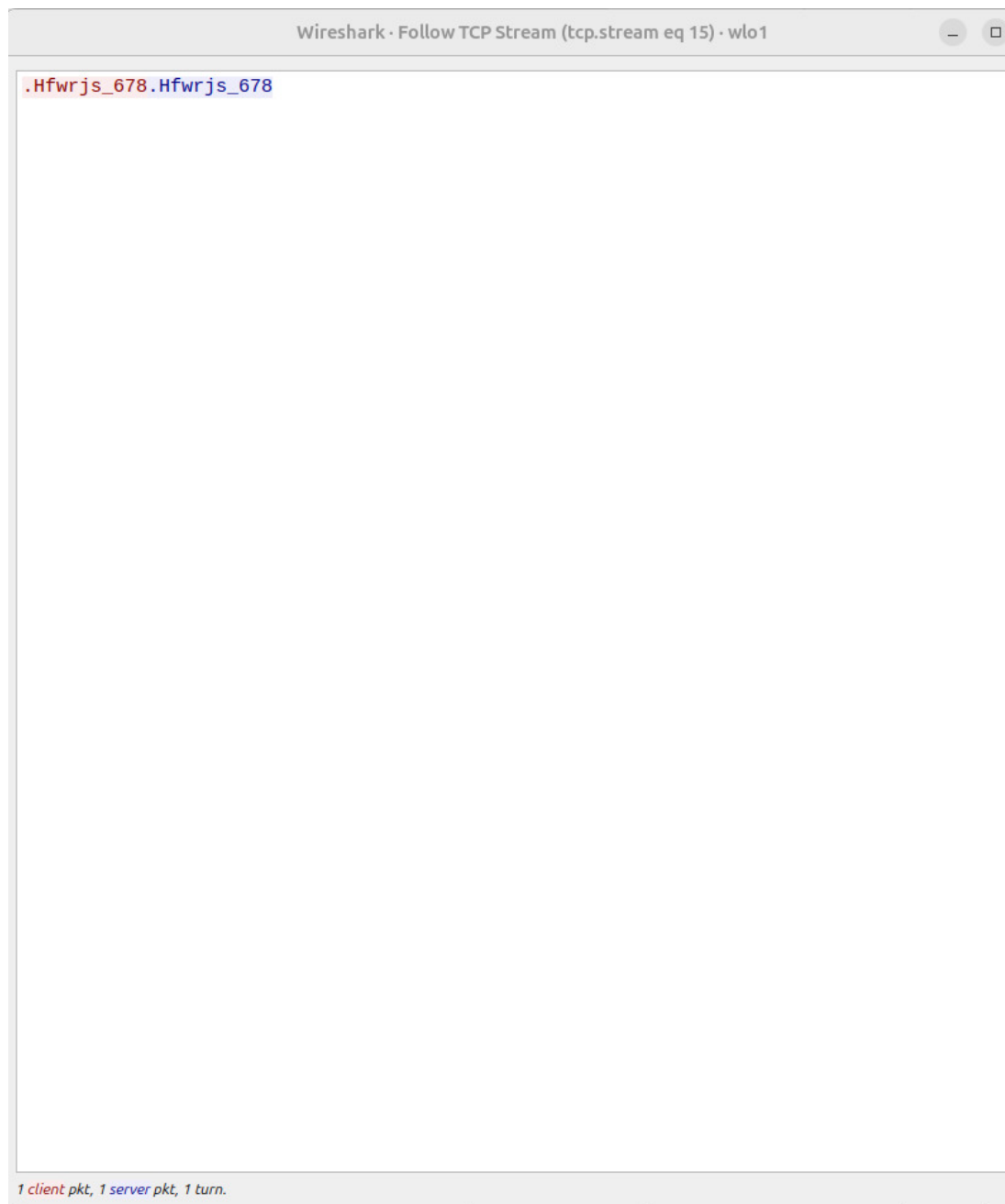


Figure 2: Wireshark packet details with encrypted payload.

3. C Code

ESP32 (Server) Main Code Snippet

```
static size_t caesar_encrypt_bytes(const char* pt, uint8_t shift,
                                   uint8_t* out, size_t cap) {
    if (!out || cap == 0) return 0;
    size_t w = 0;
    out[w++] = shift % 26;
    for (; *pt; ++pt) {
        char c = *pt;
        if (isalpha((unsigned char)c))
            c = rot_alpha(c, shift % 26);
        else if (isdigit((unsigned char)c))
            c = rot_digit(c, shift);
        if (w < cap) out[w++] = (uint8_t)c; else break;
    }
    return w;
}
```

Linux Client Code Snippet

```
int main(int argc, char *argv[]) {
    if (argc < 4) {
        printf("Usage: %s <IP> <port> <message>\n", argv[0]);
        return -1;
    }
    // TCP connection and Caesar encryption
    ...
    printf("Response %d | Decrypted: %s\n", len, message);
}
```

CMakeLists.txt (Linux)

```
cmake_minimum_required(VERSION 3.5)
project(tcp_client)
add_executable(tcp_client tcp_client.c caesar.c)
target_link_libraries(tcp_client pthread)
```

4. ESP-IDF Configuration Notes

- Example base: examples/protocols/sockets/tcp_server
- Modified Caesar cipher files: main/caesar.c, caesar.h
- Added encryption/decryption and console output.

5. Python Sniffer Output

The Python sniffer captured and decrypted the Caesar message as shown below:

```
Received 11 bytes: Encrypted data
Decrypted: Carmen_123
```

6. Validation (Caesar Cipher Tool)

Using an online Caesar cipher validation tool, the decryption was confirmed. **Shift used: 5**
Plaintext: Carmen_123
Ciphertext: Hfwjrjs_678

7. Results

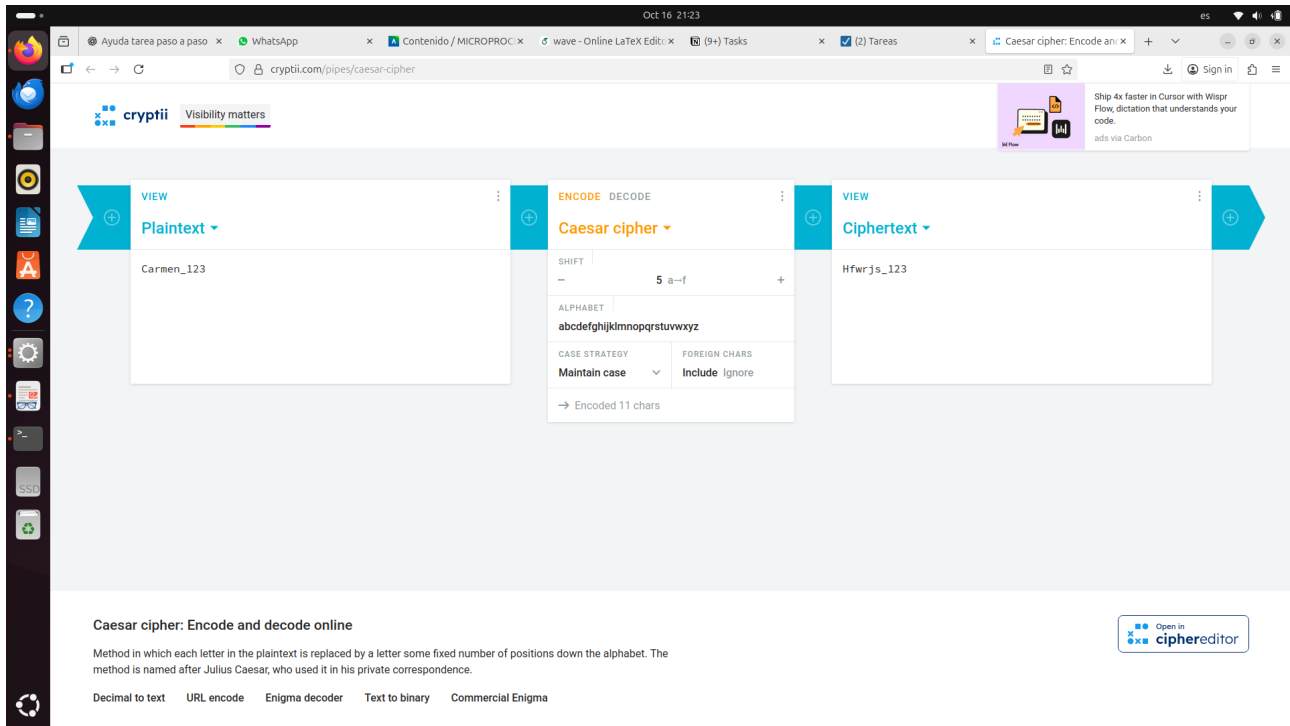


Figure 4: Caesar Cipher

Conclusion: The TCP communication between the ESP32 and the Linux client was successfully established using the Caesar cipher encryption method. The encrypted payload was correctly transmitted through the Wi-Fi network and verified in Wireshark, confirming the correct operation of both the client and server implementations. The ESP-IDF configuration allowed the ESP32 to act as a TCP server, while the Linux application connected as the client, sending and decrypting the data accurately.